

csiro-biomass

1st Place Solution

Rank	1
Team	卷不動了
Author	TheoQiu
Topic ID	670735
Version	Original

Source: <https://www.kaggle.com/competitions/csiro-biomass/discussion/670735>

Retrieved with Kaggle CLI on 2026-07-18.

Thanks to the organizers for hosting such an intense competition. We were lucky to win and I would like to thank my teammates for that. This was my first full participation in a Kaggle competition and the experience was invaluable to me. Our solutions are as follows.

CV Strategy Divide into three folds, determined jointly by **the state and sampling date**, and then train a single-fold model.

Input Data Similar to most excellent open-source notebooks, our model splits the image into left and right views and passes them through the same backbone to enhance consistency.

Training Data Augmentation We used various conventional data augmentation methods, as shown below:

```
def get_train_transforms(args):
    return Compose([
        HorizontalFlip(p=0.5),
        VerticalFlip(p=0.5),
        RandomRotate90(p=0.5),
        A.GaussNoise(p=0.3),
        A.RandomBrightnessContrast(
            brightness_limit=0.2,
            contrast_limit=0.2,
            p=0.75
        ),
        A.HueSaturationValue(
            hue_shift_limit=10,
            sat_shift_limit=20,
            val_shift_limit=20,
            p=0.5
        ),
        A.CLAHE(clip_limit=2.0, tile_grid_size=(8, 8), p=0.3),
        A.ColorJitter(
            brightness=0.2,
            contrast=0.2,
            saturation=0.2,
            hue=0.1,
            p=0.75
        ),
        Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        ),
        Resize(args.img_size, args.img_size),
        ToTensorV2()
    ])
```

After the process of splitting the left and right subimages, we apply data enhancement to the subimages independently. In addition, we apply a small size scaling to the images randomly and then complement them with black pixels to simulate the focal length and viewing angle differences of different cameras.

```
if random.random() < 0.2:
    background_image = np.full_like(image, 0)
    resize_retio = random.uniform(0.85, 1.0)
    image = cv2.resize(image, None, fx=resize_retio, fy=resize_retio, interpolation=cv2.INTER_CUBIC)
    h, w, _ = image.shape
    bg_h, bg_w, _ = background_image.shape
    top = random.randint(0, bg_h - h)
    left = random.randint(0, bg_w - w)
```

```
background_image[top:top + h, left:left + w] = image
image = background_image
```

Backbone Selection We use DINOv3 as the backbone, which is presumably the approach used by most of the winning teams in this competition. Subsequently, we use a single multi-head self-attention layer to interact with the concatenated features of the left and right sub-images, and then output the fused features through an MLP.

Head Design We independently predict the biomass of each species by using **five regression heads**, each of which contains three linear layers. No physical constraints are imposed on them during training, which we believe reduces the model's dependence on the training set.

Different from most solutions, we have additionally designed five **classification heads**. Given that this task is similar to the crowd counting (CC) problem without a density map, we made a point of reading a lot of work in the field of CC. Referring to the interval division scheme in UEPNet[1], we use the training set to divide seven intervals for each species:

```
BORDERS_DICT = {
    'Dry_Clover_g': [1.6e-05, 3.9, 10.5353, 20.6523, 37.5911, 71.7865],
    'Dry_Dead_g': [1.6e-05, 6.1407, 13.1192, 23.277, 38.8581, 83.8407],
    'Dry_Green_g': [1.6e-05, 13.4232, 27.0782, 45.5236, 79.834, 157.9836],
    'Dry_Total_g': [1.6e-05, 23.4907, 41.1, 61.1, 96.8288, 185.7],
    'GDM_g': [1.6e-05, 16.5143, 30.507, 49.5585, 81.0, 157.9836],
}
```

Based on the divided intervals, we add an additional classification head for each species to predict the interval where the biomass lies, which resulted in an improvement of approximately 0.03 in both LB and PB. This is also highly consistent with our CV. Ideally, if the intervals are divided sufficiently small, the model can accurately predict the biomass through interval inference values. (Obviously, this is too difficult for this competition without density maps/segmentation maps.)

Loss Function The model we finally submitted uses the common SmoothL1 Loss for regression and Cross-Entropy loss for classification. The loss of each biomass is scaled according to the weights proposed by the competition for the calculation of R^2 .

```
class WeightedBiomassLoss(nn.Module):
    def __init__(self, loss_weights_dict):
        super(WeightedBiomassLoss, self).__init__()

        self.criterion = nn.SmoothL1Loss()
        self.cls_loss = nn.CrossEntropyLoss(reduction='none')
        self.cls_weight = 0.3
        self.weights = loss_weights_dict

    def forward(self, predictions, targets, predictions_cls=None, cls_labels=None):
        total_cls_loss = None
        pred_green, pred_dead, pred_clover, pred_gdm, pred_total = predictions
        if predictions_cls is not None:
            assert cls_labels is not None
            pred_green_cls, pred_dead_cls, pred_clover_cls, pred_gdm_cls, pred_total_cls =
predictions_cls
            cls_green_loss = self.cls_loss(pred_green_cls, cls_labels[:, 0])
            cls_dead_loss = self.cls_loss(pred_dead_cls, cls_labels[:, 1])
```

```

        cls_clover_loss = self.cls_loss(pred_clover_cls, cls_labels[:, 2])
        cls_gdm_loss = self.cls_loss(pred_gdm_cls, cls_labels[:, 3])
        cls_total_loss = self.cls_loss(pred_total_cls, cls_labels[:, 4])
        total_cls_loss = (
            self.weights['green_loss'] * cls_green_loss.mean() +
            self.weights['dead_loss'] * cls_dead_loss.mean() +
            self.weights['clover_loss'] * cls_clover_loss.mean() +
            self.weights['gdm_loss'] * cls_gdm_loss.mean() +
            self.weights['total_loss'] * cls_total_loss.mean()
        )

    true_green = targets[:, 0].unsqueeze(-1) # Shape [batch, 1]
    true_dead = targets[:, 1].unsqueeze(-1) # Shape [batch, 1]
    true_clover = targets[:, 2].unsqueeze(-1) # Shape [batch, 1]
    true_gdm = targets[:, 3].unsqueeze(-1) # Shape [batch, 1]
    true_total = targets[:, 4].unsqueeze(-1) # Shape [batch, 1]

    loss_green = self.criterion(pred_green, true_green)
    loss_dead = self.criterion(pred_dead, true_dead)
    loss_clover = self.criterion(pred_clover, true_clover)
    loss_gdm = self.criterion(pred_gdm, true_gdm)
    loss_total = self.criterion(pred_total, true_total)

    total_loss = (
        self.weights['total_loss'] * loss_total +
        self.weights['gdm_loss'] * loss_gdm +
        self.weights['green_loss'] * loss_green +
        self.weights['dead_loss'] * loss_dead +
        self.weights['clover_loss'] * loss_clover # +
    )
    if predictions_cls is not None:
        total_loss = total_loss + self.cls_weight * total_cls_loss
    return total_loss, total_cls_loss

```

There are also some additional experiments that were not ultimately included in the submitted model, but PB has proven that our designs in this process are meaningful. For example, using epsilon-insensitive L1 loss for regression optimization. This observation is based on the uneven distribution of biomass, we hope that **"large values have large errors and small values have small errors"**. Therefore, we designed the following loss based on the label, which enabled us to achieve the best single-model result on PB.

```

class EpsilonInsensitiveLoss(nn.Module):
    def __init__(self, eps_point=20, max_eps=5, reduction='mean'):
        super().__init__()
        self.eps_point = eps_point
        self.reduction = reduction
        self.scale_ratio = 0.1
        self.max_eps = max_eps

    def make_epsilon(self, target):
        epsilon = torch.zeros_like(target)
        epsilon = torch.where(
            target <= self.eps_point, torch.ones_like(target), epsilon
        )
        epsilon = torch.where(
            target > self.eps_point, target * self.scale_ratio, epsilon
        )
        epsilon = torch.clamp(epsilon, max=self.max_eps)

        return epsilon

    def forward(self, pred, target):
        epsilon = self.make_epsilon(target.detach())

```

```

abs_diff = torch.abs(pred - target)
loss = torch.relu(abs_diff - epsilon)

if self.reduction == 'mean':
    return loss.mean()
elif self.reduction == 'sum':
    return loss.sum()
return loss

```

Training Strategy In the early stage of the competition, we tried various different strategies and finally chose the most effective two-stage training strategy.

- **Stage 1:** Freeze DINO, then train the fusion layers and MLP heads.
- **Stage 2:** Fine-tune the entire model.

Furthermore, under our setup, whether it was LB or CV, we found that the performance of DINO-Large and DINO-Huge was similar. With limited computational resources, the benefit of increasing the input size was better than increasing the model size. Therefore, in the later stages of the competition, our best single model was generally trained with the 1024 pix + large configuration.

Single model results of DINOv3-Large

exp_name	input_size (single view)	LB	PB
Baseline-large	512	0.70	0.60
+cls	512	0.73	0.63
+cls	1024	0.74	0.64
+aug_scale + cls	1024	0.74	0.65
+cls + ϵ -insensitive loss	512	0.71	0.66

Test Time Online training during testing brought us a gain of >0.02 .

- **Step 1.** We selected four single models that performed well at LB with variability to generate pseudo-labels for the test set.
- **Step 2.** Based on the obtained pseudo-labels, we iteratively trained two DINO-Large models. Different from the above training, we used the entire training set and test set for training, no longer performed CV, but instead took the SWA ensemble model of the final multiple epochs. After training with pseudo-labels, we conducted training on the training set for several epochs to prevent the model from overfitting to the pseudo-labels.
- **Step 3.** Based on the two DINO-Large models obtained in the previous step, we generated pseudo-labels for the second time, and then trained two DINO-Base models according to the same strategy.
- **Step 4.** Fusion of the inference results of the two DINO-Large models and the two DINO-Base models with weights of 0.4 and 0.6 respectively. The submission notebook has been published at [Final_Inference_Notebook](#).

Post-processing We used some simple post-processing to slightly improve the scores of LB and PB, as follows:

```
out_green = self.head_green(combined)
out_dead = self.head_dead(combined)
out_clover = self.head_clover(combined)
out_gdm = self.head_gdm(combined)
out_total = self.head_total(combined)

out_clover = out_clover * 0.8
if out_dead > 20:
    out_dead *= 1.1
elif out_dead < 10:
    out_dead *= 0.9

out_gdm = 0.5*out_gdm+ 0.5*(out_green+out_clover)

pred_total1 = out_green + out_clover + out_dead
pred_total2 = out_gdm + out_dead
out_total = 0.5*out_total + 0.5*pred_total1 + 0.0*pred_total2
```

Final Competition Score

exp_name	LB	PB
best in PB	0.77	0.68
submitted	0.77	0.67

Ideas that didn't work and Discussions

1. Various semi-supervised schemes . Implementing and tuning these methods took up almost the my last few days of the competition, but none of them performed better than direct hard semi-supervised approaches.
2. Training one model for one species and then merging five models.
3. Introducing physical constraints into training, or training several heads and then calculating the remaining biomass.
4. Removing dropout. Given that this competition is essentially a regression task, adding dropout would normally cause inconsistent variance between training and inference. However, dropout performed excellently in our experiments, significantly outperforming its removal.
5. Generate additional unlabeled data and external datasets.
6. Adding shadows to training images.
7. Adding ranking loss. Due to the small amount of training data, we originally wanted to add a ranking loss to force the model to learn to compare biomass in different regions. First, we randomly cropped the training image x to get x_{crop} , and it is certain that $y > y_{\text{crop}}$, which gives an internal ranking relationship. Then, we calculated the ranking relationships between images based on labels. This scheme performed well on the CV, once surpassing our previous best model, but got a poor LB, which might be due to overfitting again~
8. Replace L1 loss with various other losses.

9. Inputs of patches at multiple resolutions ($[1 \times 2, 2 \times 4, 4 \times 8, \dots]$). Although this scheme eventually achieved a decent CV score, it oscillated severely. Facts have also proven that their final performance on LB and PB was not good (especially PB, which got a score of <0.6).

10.

Last but not least, congratulations to my teammate [@pingfan](#), the Kaggle community is about to have a new GM !!!

[1] Wang, Changan, Qingyu Song, Boshen Zhang, Yabiao Wang, Ying Tai, Xuyi Hu, Chengjie Wang, Jilin Li, Jiayi Ma, and Yang Wu. "Uniformity in heterogeneity: Diving deep into count interval partition for crowd counting." In Proceedings of the IEEE/CVF international conference on computer vision, pp. 3234-3242. 2021.

Supplemental Q&A

V1kstrand · 2026-01-29T12:44:46.870000

Great job, team! Congratulations!

Q: How did you come to the conclusions about the " Post-processing We used some simple post-processing to slightly improve the scores of LB and PB, as follows:" step. was is done by sweeping some different settings? or rather based on some data assumptions?

TheoQiu · 2026-01-29T12:57:07.180000

I partly referenced open-source notebooks and partly derived from CV and LB scores.

benkerrouche abdelbasset · 2026-01-29T17:15:23.400000

congratulations your writeup really informative ... may i ask you questions : 1 if you use other backbone selection 2. the pseudo labeling in your last approach did you use it for stabilization and self-distillation .. thank you so much your approach really helping me

TheoQiu · 2026-01-30T13:40:25.907000

1. I also tried models from the Swin series, and my teammate tried DINOv2. Both of them are not as effective as DINOv3.
2. Yes. This is actually a self-distillation strategy.